Student performance prediction using ML

Source code :

student_performance.ipynb

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

df = pd.read_csv('/content/drive/MyDrive/sem6/Student_performance_data _.csv')
df.head()

## we just checked if there are duplicate rows or not and also for missing values , basic things

#dataset shape
print("Dataset Shape (Rows, Columns):")
print(df.shape)

print("\nColumns in Dataset:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())

print("Unique values for grade class")
print(df['GradeClass'].unique())

##data preprocessing

#dropping unneccesary columns
df2 = df.drop(columns =["StudentID"])
df2.head()

##EDA - exploratory data analysis

print("statistical summary")
print(df2.describe())
```

```
##correlation

corr = df2.corr()
print("correlation between data")
print(corr)

plt.figure(figsize=(7, 6))
plt.xticks(fontsize=8)   # reduce column name size
plt.yticks(fontsize=8)
sns.heatmap(corr, annot=True,annot_kws={"fontsize": 6},fmt=".2f")
plt.show()

## graphs for understanding EDA

sns.set(style="whitegrid")

sns.histplot(df['GPA'], bins=30, kde=True)
plt.title("Distribution of GPA")
plt.show()

sns.scatterplot(x='StudyTimeWeekly', y='GPA', data=df2)
plt.title("Study Time vs GPA")
plt.show()

sns.scatterplot(x='Absences', y='GPA', data=df2)
plt.title("Absences vs GPA")
plt.show()

sns.boxplot(x='ParentalSupport', y='GPA', data=df2)
plt.title("Parental Support vs GPA")
plt.show()

sns.boxplot(x='Tutoring', y='GPA', data=df2)
plt.title("Tutoring vs GPA")
plt.show()

sns.boxplot(x='Extracurricular', y='GPA', data=df)
plt.title("Extracurricular Activities vs GPA")
plt.show()

sns.boxplot(x='Sports', y='GPA', data=df)
plt.title("Sports Participation vs GPA")
plt.show

import seaborn as sns
import matplotlib.pyplot as plt
```

```
# graphs for understanding EDA of GradeClass

sns.set(style="whitegrid")

# GradeClass distribution
sns.countplot(x='GradeClass', data=df2)
plt.title("Distribution of GradeClass")
plt.show()

# Study Time vs GradeClass
sns.boxplot(x='GradeClass', y='StudyTimeWeekly', data=df2)
plt.title("Study Time vs GradeClass")
plt.show()

# Absences vs GradeClass
sns.boxplot(x='GradeClass', y='Absences', data=df2)
plt.title("Absences vs GradeClass")
plt.show()

# Parental Support vs GradeClass
sns.boxplot(x='GradeClass', y='ParentalSupport', data=df2)
plt.title("Parental Support vs GradeClass")
plt.show()

sns.countplot(x='GradeClass', hue='Tutoring', data=df2)
plt.title("Grade Class Distribution by Tutoring")
plt.xlabel("Grade Class (0=A, 1=B, 2=C, 3=D, 4=F)")
plt.ylabel("Count of Students")
plt.show()

sns.countplot(x='GradeClass', hue='Extracurricular', data=df2)
plt.title("Grade Class Distribution by Extracurricular Activities")
plt.xlabel("Grade Class (0=A, 1=B, 2=C, 3=D, 4=F)")
plt.ylabel("Count of Students")
plt.show()

sns.countplot(x='GradeClass', hue='Sports', data=df2)
plt.title("Grade Class Distribution by Sports Participation")
plt.xlabel("Grade Class (0=A, 1=B, 2=C, 3=D, 4=F)")
plt.ylabel("Count of Students")
plt.show()

##train_test_split
df2.head()

#train_test_split
X = df2.drop(columns=['GPA','GradeClass'])
y = df2['GPA']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train)
print(X_test)
print(y_train)
print(y_test)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

#training data error
y_train_pred = model.predict(X_train_scaled)

mse_train = mean_squared_error(y_train, y_train_pred)
mae_train = mean_absolute_error(y_train, y_train_pred)
r2_train = r2_score(y_train, y_train_pred)
rmse_train = np.sqrt(mse_train)

print("Training Data:")
print("MSE:", mse_train)
print("MAE:", mae_train)
print("R2 Score:", r2_train)
print("RMSE:", rmse_train)

plt.scatter(y_test, y_pred)
plt.xlabel("Actual GPA")
plt.ylabel("Predicted GPA")
plt.title("Actual vs Predicted GPA")
plt.show()

new_student = np.array([[18, 1, 2, 4, 10.456, 0, 1, 0, 1, 1, 1, 1]])
new_student_scaled = scaler.transform(new_student)

predicted_gpa = model.predict(new_student_scaled)
print("Predicted GPA for the new student:", predicted_gpa[0])

# Load dataset
```

```
df = pd.read_csv("/content/drive/MyDrive/sem6/student_performance_balanced.csv")
df = df.drop(columns =["StudentID"])
# ================================
# STEP 1: Check GPA Distribution
# ================================
plt.figure(figsize=(8,5))
sns.histplot(df['GPA'], bins=20, kde=True)
plt.title("Original GPA Distribution")
plt.xlabel("GPA")
plt.ylabel("Count")
plt.show()

# ================================
# STEP 2: Create GPA Categories
# ================================
# Low: GPA < 2.0
# Medium: 2.0 <= GPA < 3.0
# High: GPA >= 3.0

bins = [0, 2.0, 3.0, 4.0]
labels = ['Low', 'Medium', 'High']

df['GPA_Category'] = pd.cut(df['GPA'], bins=bins, labels=labels, include_lowest=True)

# Show counts
print("GPA Category Counts:")
print(df['GPA_Category'].value_counts())

# Plot category distribution
plt.figure(figsize=(6,4))
sns.countplot(x='GPA_Category', data=df)
plt.title("GPA Category Distribution")
plt.xlabel("GPA Category")
plt.ylabel("Count")
plt.show()

# ================================
# STEP 3: Calculate Sample Weights
# ================================
# Inverse frequency weighting
category_counts = df['GPA_Category'].value_counts().to_dict()

df['sample_weight'] = df['GPA_Category'].map(lambda x: 1 / category_counts[x])

# Normalize weights so average weight = 1
df['sample_weight'] = pd.to_numeric(df['sample_weight'], errors='coerce')
df['sample_weight'] = df['sample_weight'] / df['sample_weight'].mean()
```

```
# ================================
# STEP 4: Check Balanced Weights
# ================================
print("\nSample Weights Assigned:")
print(df[['GPA', 'GPA_Category', 'sample_weight']].head(10))

# Average weight per category
print("\nAverage Weight per GPA Category:")
print(df.groupby('GPA_Category')['sample_weight'].mean())

# ================================
# STEP 5: Save Balanced Dataset
# ================================
df.to_csv("student_performance_balanced.csv", index=False)

print("\nBalanced dataset saved as 'student_performance_balanced.csv'")

# ========================================
# STEP 1: IMPORT LIBRARIES
# ========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# ========================================
# STEP 2: LOAD BALANCED DATASET
# ========================================
df = pd.read_csv("/content/student_performance_balanced.csv")   # or your saved path

print("Dataset Shape:", df.shape)
print("\nColumns:\n", df.columns)

# ========================================
# STEP 3: DEFINE FEATURES AND TARGET
# ========================================
# Target = GPA
y = df['GPA']
```

```
# Remove unnecessary columns
X = df.drop(columns=['StudentID', 'GPA', 'GradeClass', 'GPA_Category', 'sample_weight'], errors='ignore')

# Sample weights
sample_weights = df['sample_weight']

print("\nFeature Columns Used:")
print(X.columns.tolist())

# ==========================================
# STEP 4: IDENTIFY NUMERICAL AND CATEGORICAL COLUMNS
# ==========================================
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()

print("\nNumerical Columns:", numerical_cols)
print("Categorical Columns:", categorical_cols)

# ==========================================
# STEP 5: PREPROCESSING
# ==========================================
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean'))
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numerical_cols),
    ('cat', categorical_transformer, categorical_cols)
])

# ==========================================
# STEP 6: TRAIN-TEST SPLIT
# ==========================================
X_train, X_test, y_train, y_test, w_train, w_test = train_test_split(
    X, y, sample_weights, test_size=0.2, random_state=42
)

print("\nTrain Shape:", X_train.shape)
print("Test Shape:", X_test.shape)

# ==========================================
# STEP 7: LINEAR REGRESSION MODEL
# ==========================================
```

```
linear_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])

linear_model.fit(X_train, y_train, model__sample_weight=w_train)

y_pred_lr = linear_model.predict(X_test)

# Metrics for Linear Regression
mae_lr = mean_absolute_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("\n===== Linear Regression Results =====")
print("MAE :", mae_lr)
print("MSE :", mse_lr)
print("RMSE:", rmse_lr)
print("R² Score:", r2_lr)

plt.scatter(y_test, y_pred_lr)
plt.xlabel("Actual GPA")
plt.ylabel("Predicted GPA")
plt.title("Actual vs Predicted GPA")
plt.show()

# ========================================
# STEP 8: RANDOM FOREST REGRESSOR MODEL
# ========================================
rf_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(
        n_estimators=200,
        max_depth=10,
        random_state=42
    ))
])

rf_model.fit(X_train, y_train, model__sample_weight=w_train)

y_pred_rf = rf_model.predict(X_test)

# Metrics for Random Forest
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)
```

```
print("\n===== Random Forest Regressor Results =====")
print("MAE :", mae_rf)
print("MSE :", mse_rf)
print("RMSE:", rmse_rf)
print("R² Score:", r2_rf)

plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual GPA")
plt.ylabel("Predicted GPA")
plt.title("Actual vs Predicted GPA")
plt.show()

# ==========================================
# STEP 9: COMPARE BOTH MODELS
# ==========================================
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest Regressor'],
    'MAE': [mae_lr, mae_rf],
    'MSE': [mse_lr, mse_rf],
    'RMSE': [rmse_lr, rmse_rf],
    'R2 Score': [r2_lr, r2_rf]
})

print("\n===== Model Comparison =====")
print(results)

# ==========================================
# STEP 10: BAR CHART FOR COMPARISON
# ==========================================
plt.figure(figsize=(8,5))
sns.barplot(x='Model', y='R2 Score', data=results)
plt.title("Model Comparison based on R² Score")
plt.ylabel("R² Score")
plt.xlabel("Model")
plt.show()

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd

# ==========================================
# CHECK BALANCED DATASET COLUMNS
# ==========================================
print("Balanced Dataset Columns:")
print(df.columns)

print("\nFirst 5 Rows of Balanced Dataset:")
print(df.head())
```

```
# ==========================================
# DECISION TREE REGRESSOR MODEL
# ==========================================
dt_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', DecisionTreeRegressor(
        max_depth=10,
        random_state=42
    ))
])

# Train model using balanced sample weights
dt_model.fit(X_train, y_train, model__sample_weight=w_train)

# Predict on test data
y_pred_dt = dt_model.predict(X_test)

# ==========================================
# EVALUATION METRICS
# ==========================================
mae_dt = mean_absolute_error(y_test, y_pred_dt)
mse_dt = mean_squared_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mse_dt)
r2_dt = r2_score(y_test, y_pred_dt)

print("\n===== Decision Tree Regressor Results =====")
print("MAE :", mae_dt)
print("MSE :", mse_dt)
print("RMSE:", rmse_dt)
print("R² Score:", r2_dt)

# ==========================================
# UPDATED MODEL COMPARISON TABLE
# ==========================================
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Decision Tree Regressor', 'Random Forest Regressor'],
    'MAE': [mae_lr, mae_dt, mae_rf],
    'MSE': [mse_lr, mse_dt, mse_rf],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf],
    'R2 Score': [r2_lr, r2_dt, r2_rf]
})

print("\n===== Updated Model Comparison =====")
print(results)

plt.figure(figsize=(8,5))
sns.barplot(x='Model', y='R2 Score', data=results)
plt.title("Model Comparison based on R² Score")
```

```
plt.ylabel("R² Score")
plt.xlabel("Model")
plt.show()
```

description :

in our project we have taken a dataset and gone through Machine Learning life cycle where steps are like

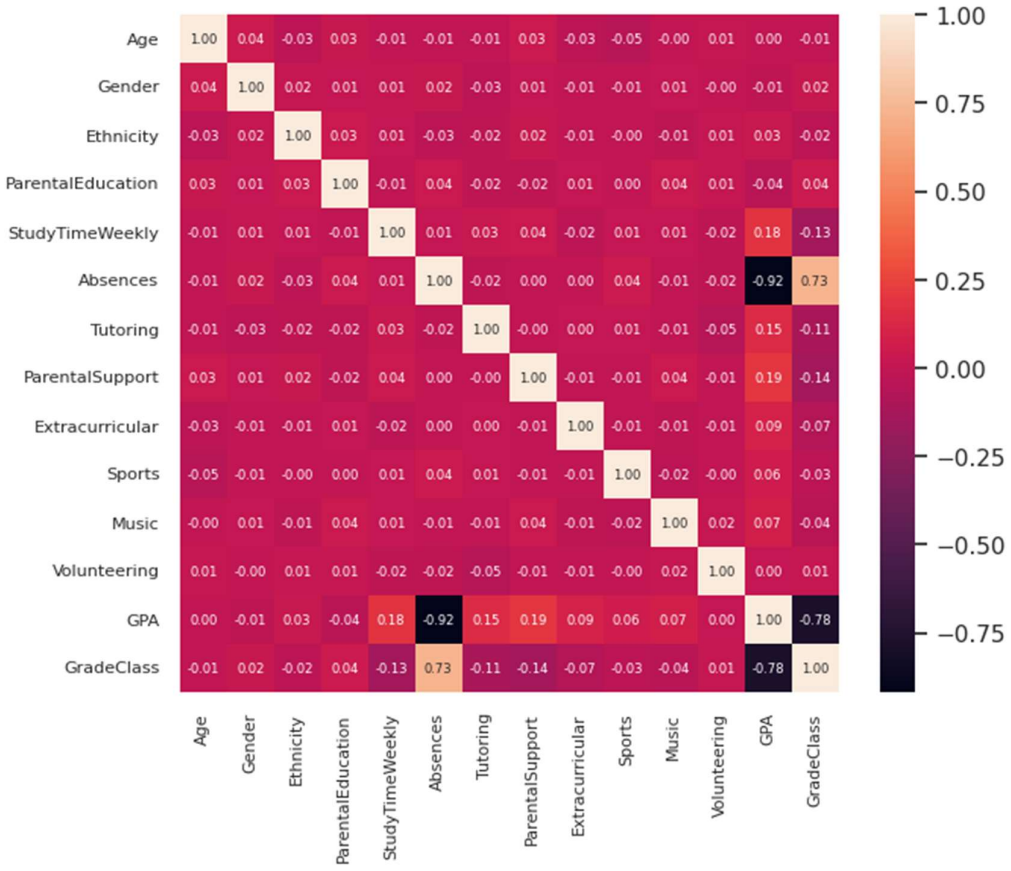
1.data gathering
2.data cleaning
3.data preprocessing
4.Exploratory data analysis
5.train test split
6.model implementation
7.prediction
8.error calculation
9.model evaluation

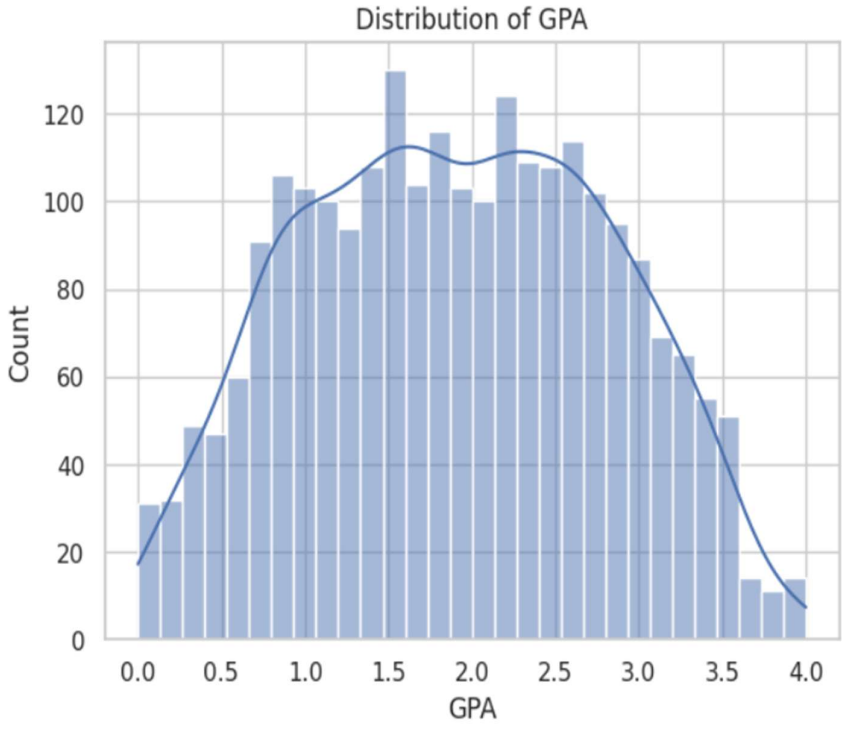
This is how our dataset looks life before balancing (in normal form):

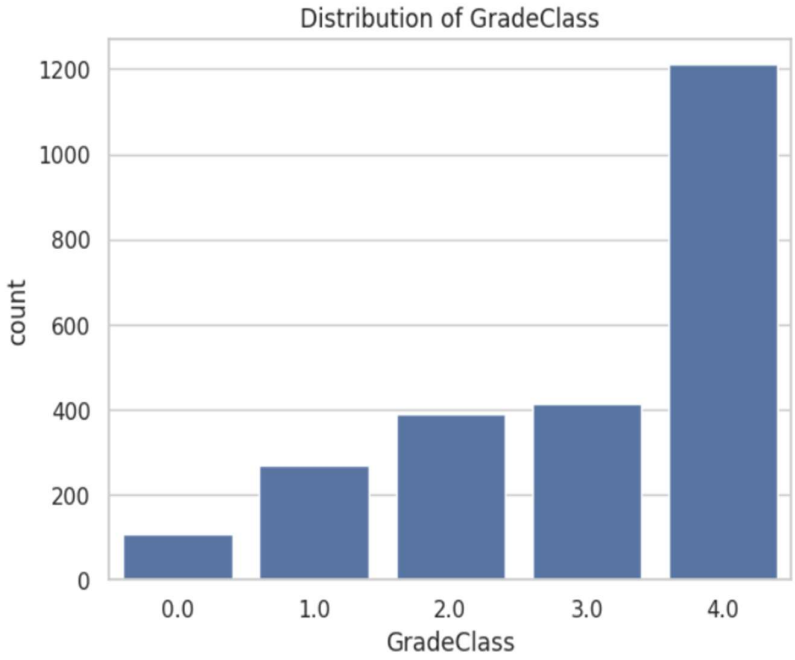
Age	Gender	Ethnicity	Parental Education	Study Time Weekly	Absences	Tutoring	ParentalSupport	Extracurricular	Sports	Music	Volunteering	GPA	Grade Class
17	1	0	2	19.833723	7	1	2	0	0	1	0	2.929196	2.0
18	0	0	1	15.408756	0	0	1	0	0	0	0	3.042915	1.0
15	0	2	3	4.210570	26	0	2	0	0	0	0	0.112602	4.0
17	1	0	3	10.028829	14	0	3	1	0	0	0	2.054218	3.0
17	1	0	2	4.672495	17	1	3	0	0	0	0	1.288061	4.0

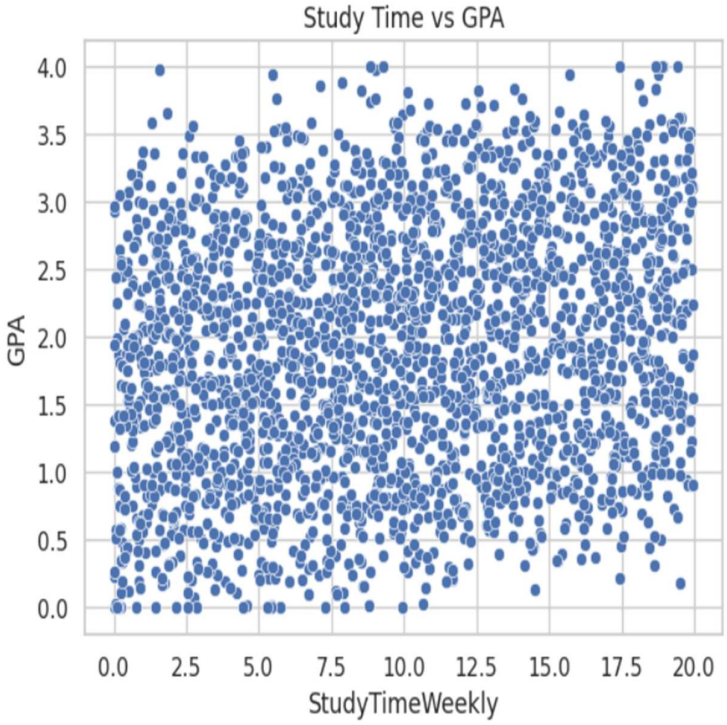
Target variable : GPA
Independent variable : all other except grade_class

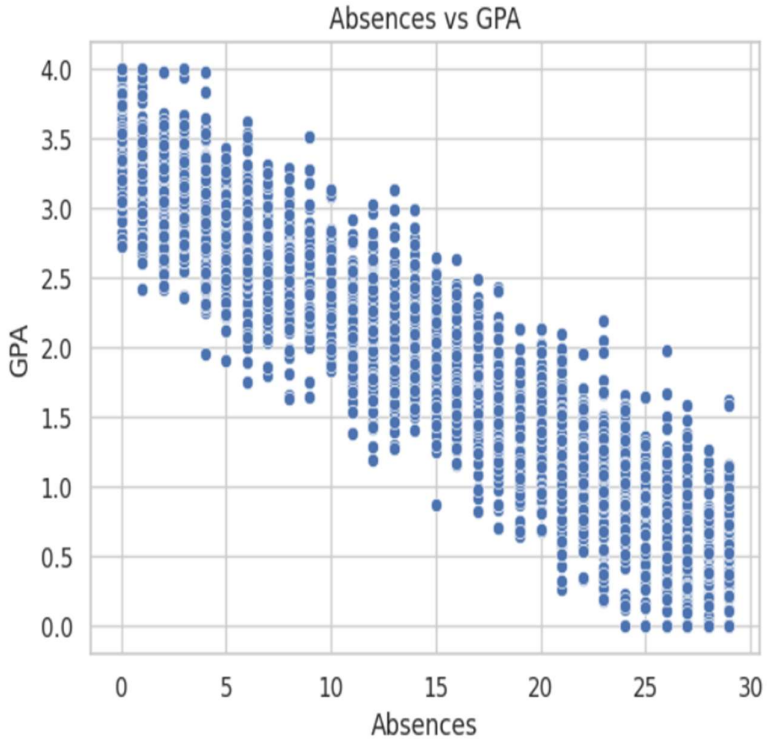
EDA:

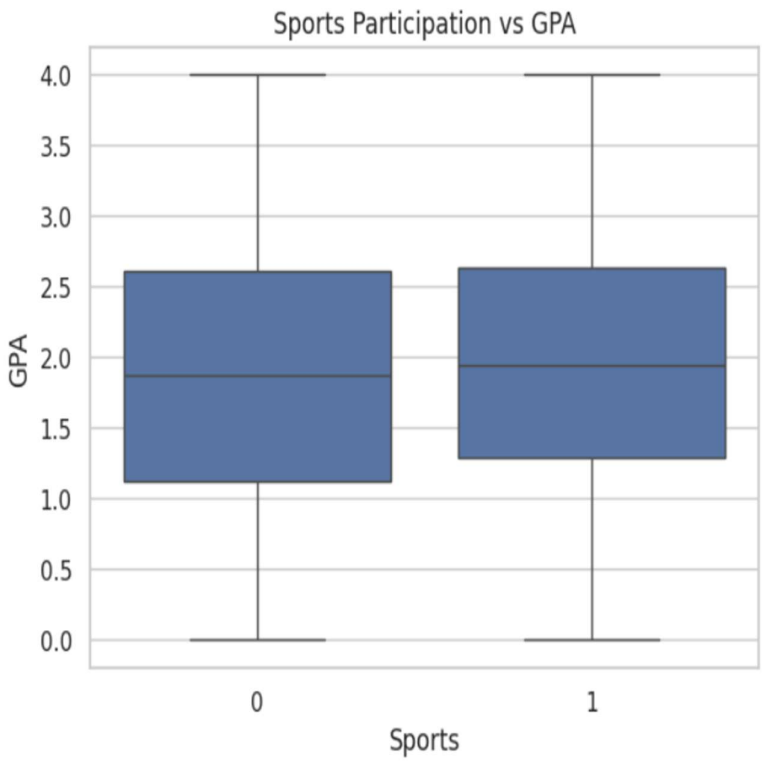

Age 1.00 0.04 -0.03 0.03 -0.01 -0.01 -0.01 0.03 -0.03 -0.05 -0.00 0.01 0.00 -0.01
Gender 0.04 1.00 0.02 0.01 0.01 0.02 -0.03 0.01 -0.01 -0.01 0.01 -0.00 -0.01 0.02
Ethnicity -0.03 0.02 1.00 0.03 0.01 -0.03 -0.02 0.02 -0.01 -0.00 -0.01 0.01 0.03 -0.02
ParentalEducation 0.03 0.01 0.03 1.00 -0.01 0.04 -0.02 -0.02 0.01 0.00 0.04 0.01 -0.04 0.04
StudyTimeWeekly -0.01 0.01 0.01 -0.01 1.00 0.01 0.03 0.04 -0.02 0.01 0.01 -0.02 0.18 -0.13
Absences -0.01 0.02 -0.03 0.04 0.01 1.00 -0.02 0.00 0.00 0.04 -0.01 -0.02 -0.92 0.73
Tutoring -0.01 -0.03 -0.02 -0.02 0.03 -0.02 1.00 -0.00 0.00 0.01 -0.01 -0.05 0.15 -0.11
ParentalSupport 0.03 0.01 0.02 -0.02 0.04 0.00 -0.00 1.00 -0.01 -0.01 0.04 -0.01 0.19 -0.14
Extracurricular -0.03 -0.01 -0.01 0.01 -0.02 0.00 0.00 -0.01 1.00 -0.01 -0.01 -0.01 0.09 -0.07
Sports -0.05 -0.01 -0.00 0.00 0.01 0.04 0.01 -0.01 -0.01 1.00 -0.02 -0.00 0.06 -0.03
Music -0.00 0.01 -0.01 0.04 0.01 -0.01 -0.01 0.04 -0.01 -0.02 1.00 0.02 0.07 -0.04
Volunteering 0.01 -0.00 0.01 0.01 -0.02 -0.02 -0.05 -0.01 -0.01 -0.00 0.02 1.00 0.00 0.01
GPA 0.00 -0.01 0.03 -0.04 0.18 -0.92 0.15 0.19 0.09 0.06 0.07 0.00 1.00 -0.78
GradeClass -0.01 0.02 -0.02 0.04 -0.13 0.73 -0.11 -0.14 -0.07 -0.03 -0.04 0.01 -0.78 1.00
Age Gender Ethnicity ParentalEducation StudyTimeWeekly Absences Tutoring ParentalSupport Extracurricular Sports Music Volunteering GPA GradeClass
1.00 0.75 0.50 0.25 0.00 −0.25 −0.50 −0.75

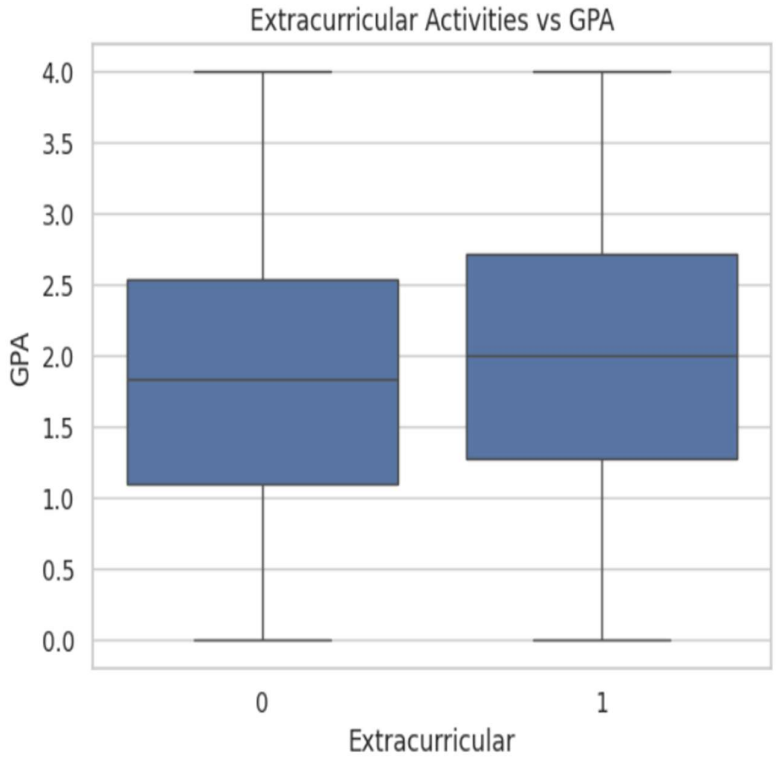

Distribution of GPA
Count
0 20 40 60 80 100 120
0.0 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0
GPA


Distribution of GradeClass
count
1200
1000
800
600
400
200
0
0.0
1.0
2.0
3.0
4.0
GradeClass

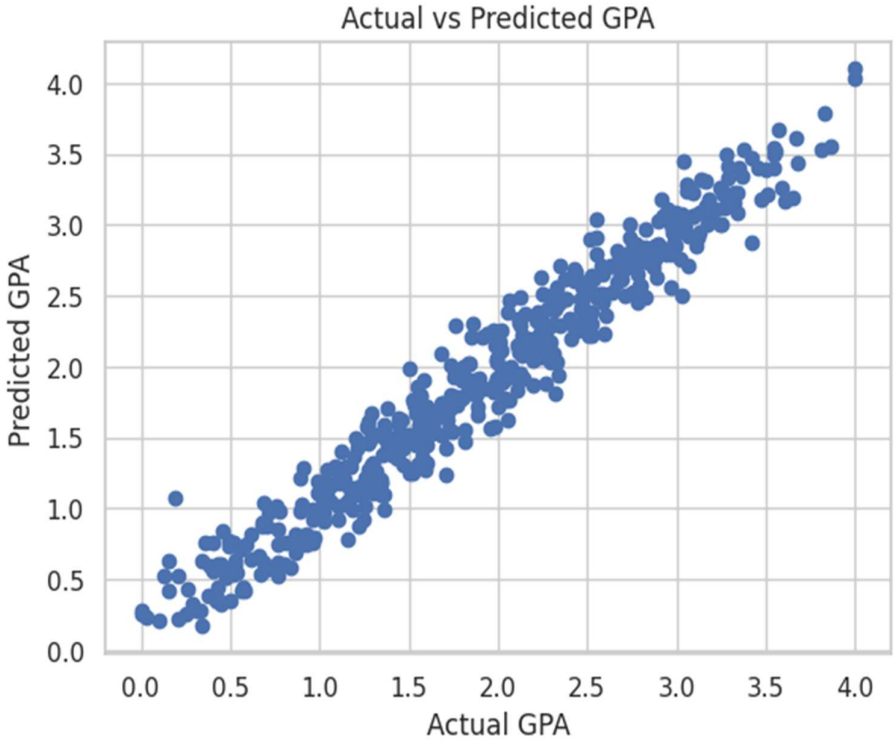

Study Time vs GPA
GPA
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
StudyTimeWeekly


Absences vs GPA
GPA
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
5
10
15
20
25
30
Absences


Sports Participation vs GPA
GPA
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
Sports



This is the difference between actual and predicted values in testing :



Prediction by giving input to values :

```
new_student = np.array([[18, 1, 2, 4, 10.456, 0, 1, 0, 1, 1, 1, 1]])
new_student_scaled = scaler.transform(new_student)

predicted_gpa = model.predict(new_student_scaled)
print("Predicted GPA for the new student:", predicted_gpa[0])
```

output :

Predicted GPA for the new student: 3.606038519252732

Accuracy :

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	0.156015	0.039166	0.197903	0.952638
1	Decision Tree Regressor	0.262905	0.110157	0.331899	0.866789
2	Random Forest Regressor	0.187598	0.058413	0.241687	0.929362

GitHub repository link :

https://github.com/darwin08-os/Student_performance_using_ML.git

Conclusion :

As we can see that now our model is able to predict the new student GPA based on data so that we can identify which student need academic support and those who are performing well , we can plan accordingly for their future and we came to know that linear regression is out best model because it is giving high accuracy (R2 score) as 95% and we can use it in real time also.